Projektrapport – BilCirkeln "Vi köper din bil"

Examensprojekt, fullstack .NET, 4 veckor, individuellt.

Källkod: https://github.com/hassan-jawdat/BilCirkeln

1. Bakgrund och problemformulering

Att sälja bil privat tar tid. Säljaren måste fotografera bilen, skriva en annons, svara på frågor och boka provkörningar, och ofta prutar köparen i sista stund. Många vill hellre sälja direkt till en handlare, men då sker kontakten ofta via telefon och mejl. För handlaren blir det svårt att hålla ordning på vilka bilar som har fått bud och vad kunden har svarat.

Problemformulering: Hur kan en enkel webbapplikation göra det lätt för en privatperson att få ett bud på sin bil, och samtidigt ge personalen en tydlig överblick över alla förfrågningar?

2. Mål och avgränsningar

Mål

- Kunden kan registrera sig, skicka in en bil och följa statusen
- Personalen kan se alla förfrågningar, filtrera dem och lämna bud
- Kunden kan acceptera eller neka budet
- Säker inloggning med roller
- Affärsreglerna är testade med automatiska tester

Avgränsningar: ingen bilduppladdning, ingen betalning eller avtalssignering, ingen hämtning av bildata från Transportstyrelsen, ingen e-postavisering och ingen molndrift. Se projektplanen.

3. Vald lösning

En webbapplikation med en React-frontend som pratar med ett ASP.NET Core Web API via JSON. API:et sparar data i SQL Server via Entity Framework Core.

```
[React (Vite)]  --HTTP/JSON + JWT-->  [ASP.NET Core Web API]  --EF Core-->  [SQL Server]
   sidor, formulär                     Controllers → Services              AspNetUsers, CarSubmissions, Offers
```

Datamodell

- `ApplicationUser` (Identity-användare + `FullName`)
- `CarSubmission`: bilen som kunden vill sälja (regnr, märke, modell, år, miltal, drivmedel, växellåda, önskat pris, status) och 10 extra uppgifter om bilen: antal nycklar, extra däck, servicebok, servicehistorik, besiktning, säljartyp, import, finansiering och klädselns färg och material
- `Offer`: ett bud från personalen (belopp, meddelande, status). En `CarSubmission` kan ha **flera** `Offer`.

Statusflöde

```
Pending ──(personal lämnar bud)──► OfferMade ──(kund accepterar)──► Accepted
   │                                  │
   │                                  └──(kund nekar)──► Declined ──(nytt bud)──► OfferMade
   └──(personal avböjer)──► Rejected ◄──(personal avböjer)── Declined
```

Säljformuläret visas **ett steg i taget**, 16 steg totalt. Det börjar med ett fält som ser ut som en svensk registreringsskylt. Nästa fråga visas först när den förra är besvarad, och en mätare bredvid visar hur många procent som är ifyllt. Frågor med fasta svar besvaras med knappar (Ja/Nej, färger osv.). Kunden anger miltalet i mil, som räknas om till km innan det sparas.

4. Genomförande

Arbetet följde projektplanen med prioriteringen måste/bör/kan:

1. Struktur: solution, API-projekt, testprojekt, React-app, git
2. Datamodell och `AppDbContext`, första migrationen
3. Identity + JWT + roller och seed av personalkonto
4. Services med affärsregler och tunna controllers med DTOs
5. xUnit-tester
6. Frontend: auth-context, routing, kund- och personalsidor
7. Manuell test av hela flödet, sedan dokumentation
8. Efter test i webbläsaren gjorde jag om säljformuläret. Jag inspirerades av riktiga inköpstjänster och la till skylten, steg-för-steg-visningen och procentmätaren
9. 10 extra frågor om bilen, så att personalen kan lämna ett mer korrekt bud. Det krävde en ny migration (`AddCarDetails`)
10. Koden publicerades på GitHub och projektet redovisades muntligt

Detaljer finns i projektdagboken och beslutsloggen.

5. Tekniska val och motiveringar

Val	Motivering
ASP.NET Core Web API (.NET 8 LTS)	Kursens fokus. Långtidssupport
EF Core code first + migrations	Databasen skapas från C#-klasserna, och ändringar versionshanteras
Identity	Färdig och beprövad lösning för användare och lösenordshashning, så man slipper skriva egen krypto
JWT	API:et blir stateless, och samma token fungerar för React-appen
Services + tunna controllers	Affärslogiken går att testa utan HTTP, och controllers blir lätta att läsa
DTOs	Vi bestämmer exakt vilken data som går in och ut, och läcker inga interna fält
`ServiceResult<T>`	Förväntade fel (hittas inte, ingen behörighet, ogiltig status) hanteras utan exceptions
React + Vite + React Router + Axios	Snabb utvecklingsmiljö, sidnavigering och enkel HTTP med interceptors
xUnit + InMemory-databas	Snabba tester av affärsreglerna utan SQL Server
Nya kolumner nullable i databasen, men `[Required]` i DTO:n	Äldre förfrågningar saknar svaren och ska ändå fungera. Nya förfrågningar måste ha svar
REST API med JSON över HTTP	Standardsättet för en React-frontend att prata med en backend

6. Säkerhetsaspekter

- **Lösenord** hashas av Identity (PBKDF2) och sparas aldrig i klartext. Regel: minst 8 tecken, stor och liten bokstav och siffra.
- **JWT** signeras med HMAC-SHA256. API:et kontrollerar utfärdare, mottagare, giltighetstid (2 h) och signatur.
- **Hemligheter** (JWT-nyckel, connection string, personalens lösenord) ligger i *user secrets*, inte i git.
- **Behörighet:** `[Authorize(Roles = ...)]` på endpoints. Dessutom kontrollerar servicen att en kund bara kan se och svara på sina **egna** bilar (403 annars).
- **Användar-id tas från token**, inte från frontend, så man kan inte utge sig för att vara någon annan.
- **Registrering ger alltid rollen Kund**, så ingen kan göra sig själv till personal.
- **Samma felmeddelande** vid fel e-post och fel lösenord, så att en angripare inte kan ta reda på vilka konton som finns.
- **Validering** på servern med DataAnnotations (regnr-format, årtal, pris, längd på text). Frontend-valideringen är bara för användarvänlighet.
- **CORS** tillåter bara frontendens adress.
- **SQL-injektion** förhindras eftersom EF Core använder parametriserade frågor.
- **XSS:** React escapar all text automatiskt.
- *Kända svagheter:* appen körs lokalt över **HTTP, inte HTTPS/TLS**. Det är okej på `localhost`, eftersom trafiken aldrig lämnar datorn. I en riktig miljö skulle lösenord och JWT-token annars skickas i klartext, så där måste allt gå över HTTPS (`UseHttpsRedirection` + HSTS). Token i `localStorage` kan stjälas vid en XSS-attack. Det finns ingen rate limiting på inloggningen och ingen utloggning på servern (token gäller tills den går ut).

7. Testning och resultat

Automatiska tester (xUnit): 18 tester, alla godkända (`dotnet test`).

Område	Exempel på testfall
Skapa förfrågan	Status blir `Pending`, regnr normaliseras ("abc 123" → "ABC123"), dubblett nekas, extra bildetaljer sparas
Behörighet	Kund ser bara sina egna. Annan kund får `Forbidden`. Personal ser alla
Bud	Bud på `Pending` ger `OfferMade`. Två öppna bud samtidigt nekas. Okänt id ger `NotFound`
Svar på bud	Acceptera ger `Accepted`. Neka och sedan nytt bud fungerar. Svar utan bud nekas. Annan kund nekas
Avböja	`Pending` → `Rejected`. Accepterad bil kan inte avböjas
JWT	Token innehåller id, e-post och roll och har rätt giltighetstid

Manuella tester (Swagger/HTTP + webbläsare)

Test	Förväntat	Resultat
Anrop utan token	401	✅
Kund anropar personallistan	403	✅
Fel lösenord	400 + "Fel e-post eller lösenord."	✅
Ogiltigt regnr "X1"	400 + valideringsmeddelande	✅
Hela flödet bud → neka → nytt bud → acceptera	Statusen följer flödet	✅
Bud efter accepterad	400	✅
CORS från annan origin	Ingen `Access-Control-Allow-Origin`	✅

Resultat: Alla krav under *måste ha* och *bör ha* i projektplanen är uppfyllda.

8. Användning av AI

AI-verktyget **Claude Code** har använts för att:

- ta fram en första version av backend, frontend, tester och dokumentation utifrån mina krav
- förklara begrepp (DbContext, JWT, dependency injection, CORS m.m.), se kodgenomgang.md

Hur jag kontrollerade AI:ns arbete:

- byggde projektet och körde alla tester (18/18)
- testade alla endpoints, både rätt och fel fall, och kontrollerade statuskoder
- testade appen själv i webbläsaren som både kund och personal, och hela flödet från inskickad bil till accepterat bud
- gick igenom koden och lät AI:n förklara begrepp som jag inte förstod (JWT, REST, HTTP/TLS, dependency injection, databasens relationer) tills jag kunde förklara dem själv

Vad jag själv bestämde:

- Hur säljformuläret skulle se ut. Jag tog inspiration från en riktig inköpstjänst och ville ha ett fält som ser ut som en registreringsskylt, frågor som visas en i taget och en mätare som visar hur långt man har kommit.
- Att formuläret skulle ha fler frågor om bilen (nycklar, däck, service, besiktning, klädsel med mera), så att personalen kan ge ett bättre bud.

Vad jag lärde mig: hur frontend och backend pratar med varandra via HTTP och JSON, hur inloggning med JWT och roller fungerar, och hur databasen byggs från C#-klasser med Entity Framework och migrations.

Reflektion om AI: AI gjorde att jag kunde bygga mer än jag hade hunnit själv. Nackdelen är att det går fort, och då är det lätt att ha kod som man inte förstår. Jag fick lägga mycket tid på att gå igenom och förstå koden i efterhand.

9. Problem och utmaningar

- `AddIdentity` slår på cookie-inloggning som standard och krockar med JWT. Lösningen blev `AddIdentityCore` + `AddRoles`.
- SQL Server tillåter inte två cascade-delete-vägar till samma tabell (`Offer → User` både via bilen och via personalen). Lösningen blev `DeleteBehavior.Restrict` på personalrelationen.
- Teckenkodning vid manuella test i terminalen ("ä" skickades fel). Problemet låg i verktyget, inte i appen.
- När de 10 nya frågorna lades till fanns redan bilar i databasen som saknade svaren. Om kolumnerna hade krävt ett värde skulle databasen ha fått hitta på svar åt dem. Lösningen blev nullable kolumner, och gamla bilar visar "–".
- Med drivmedel och växellåda förvalda räknades steget som klart direkt och hoppades över. Lösningen blev att de börjar tomma ("Välj...").
- **Att förstå all kod.** Det svåraste för mig var att kunna förklara allt som byggdes, till exempel JWT, skillnaden mellan HTTP och HTTPS och hur lagren i backend hänger ihop. Jag löste det genom att be om förklaringar och kontrollfrågor tills jag kunde svara med egna ord.
- **Tidsplanen.** Jag la mer tid på att bygga och förbättra appen än på dokumentationen. Därför blev GitHub-inlämningen sen, och projektdagboken skrevs inte efter varje arbetspass.

10. Reflektion och vidareutveckling

Vad gick bra: Appen fungerar som det var tänkt. En kund kan sälja sin bil, personalen kan lämna bud och kunden kan svara. Jag är särskilt nöjd med säljformuläret, som blev tydligt och enkelt att använda. Alla 18 tester går igenom, och demon fungerade vid redovisningen.

Vad var svårt: Att förstå säkerhetsdelarna, som JWT, hashning av lösenord och TLS, och att hålla dokumentationen uppdaterad samtidigt som jag byggde.

Vad jag skulle göra annorlunda:

- Skriva i projektdagboken efter varje arbetspass i stället för i efterhand.
- Pusha till GitHub tidigt och ofta, så att inlämningen aldrig blir sen.
- Bygga i mindre steg och förstå varje del innan jag går vidare till nästa.
- Använda HTTPS från början, även lokalt.

Begränsningar

- Inga bilder av bilen
- Personalen kan inte se statistik
- Token kan inte återkallas före utgångstiden
- Körs bara lokalt och utan HTTPS
- Om två i personalen lämnar bud exakt samtidigt kan båda sparas (race condition)

Möjlig vidareutveckling

- Bilduppladdning (t.ex. Azure Blob Storage)
- Hämta bildata automatiskt via regnr
- E-postavisering när ett bud lämnas eller besvaras
- Integrationstester med `WebApplicationFactory`
- Refresh tokens och token i HttpOnly-cookie
- Driftsättning i Azure med HTTPS